Графы

Неориентированные графы, определения, представления, алгоритмы обхода (dfs и bfs), минимальное основное дерево

Определения

Вершина, ребро, инцидентность, смежность

Вершина, Ребро, Граф

V — Вершинами называется произвольное непустое множество.

$E \subseteq \{V \times V\}$ — Рёбрами называется множество неупорядоченных пар различных вершин.

$G = \{V, E\}$ — Графом называется совокупность множества вершин и множества рёбер.

$V \equiv G(V), E \equiv G(E)$

Подграфом графа G называется граф $\acute{G}$, множество вершин которого является подмножеством вершин $V(G)$ и множество рёбер является подмножеством рёбер $E(G)$.

Ицидентность (примыкание) и смежность

Вершина u и ребро e называются инцидентными, если u является элементом из пары e.

Две вершины называются смежными, если они инцидентны одному ребру.

Два ребра называются смежными, если они инцидентны одной вершине.

Степенью вершины называется количество инцидентных рёбер.

Обобщения графов

Мультиграфом называется граф с кратными рёбрами (несколько рёбер соединяют одну и ту же пару вершин).

Псевдографом называется граф с рёбрами, которые начинаются и заканчиваются в одной и той же вершине (граф с петлями).

Гипергрфом называется граф, в котором рёбра определяются как множества инцидентных вершин.

Способы задания графа

Перечисление, изображение, списки инцидентности, матрица смежности

Перечисление

V_1={a, b, c, d}
E_1={{a,b},{b,c},{c,d}, {a,c},{b,d}}

V_2={12, 6, 2, 3, 1}
E_2={{12,6}, {12,3}, {12,2}, {12,1}, {6,2}, {6,3}, {6,1}, {2,1}, {3,1}}

V_3={Ω}
E_3={}

V_4={Красный, Жёлтый, Зелёный}
E_4={{Красный, Зелёный}}

Списки инцидентности

V_1={a, b, c, d}
E_1={{a,b},{b,c},{c,d},{d,a},{a,c},{b,d}}

{a, b}, {a, c}
{b, a}, {b, c}, {b, d}
{c, a}, {c, b}, {c, d}
{d, b}, {d, c}

V_2={12, 6, 2, 3, 1}
E_2={{12,6}, {12,3}, {12,2}, {12,1}, {6,2}, {6,3}, {6,1}, {2,1}, {3,1}}

{12, 6}, {12, 2}, {12, 3}, {12, 1}
{6, 12}, {6, 2}, {6, 3}, {6, 1}
{3, 12}, {3, 6}, {3, 1}
{2, 12}, {2, 6}, {2, 1}
{1, 12}, {1, 6}, {1, 2}, {1, 3}

V_4={Красный, Жёлтый, Зелёный}
E_4={{Красный, Зелёный}}

{Красный, Зелёный}
{}
{Зелёный, Красный}

Списки смежности

V_1={a, b, c, d}
E_1={{a,b},{b,c},{c,d},{d,a},{a,c},{b,d}}

{b, c}
{a, c, d}
{a, b, d}
{b, c}

V_2={12, 6, 2, 3, 1}
E_2={{12,6}, {12,3}, {12,2}, {12,1}, {6,2}, {6,3}, {6,1}, {2,1}, {3,1}}

{6, 2, 3, 1}
{12, 2, 3, 1}
{12, 6, 1}
{12, 6, 1}
{12, 6, 2, 3}

V_4={Красный, Жёлтый, Зелёный}
E_4={{Красный, Зелёный}}

{Зелёный}
{}
{Зелёный}

Матрица смежности

V_1={a, b, c, d}
E_1={{a,b},{b,c},{c,d},{a,c},{b,d}}

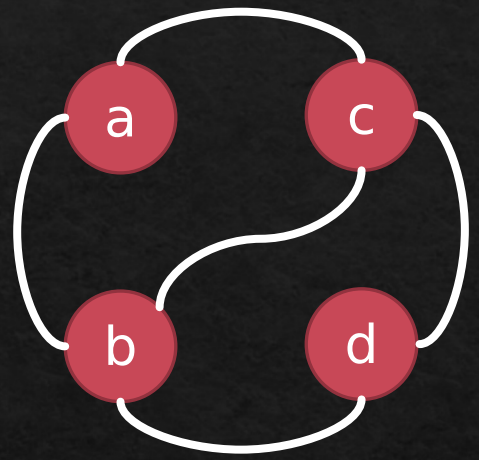
	a	b	c	d
a		+	+	
b	+		+	+
c	+	+		+
d		+	+	

V_4={Красный, Жёлтый, Зелёный}
E_4={{Красный, Зелёный}}

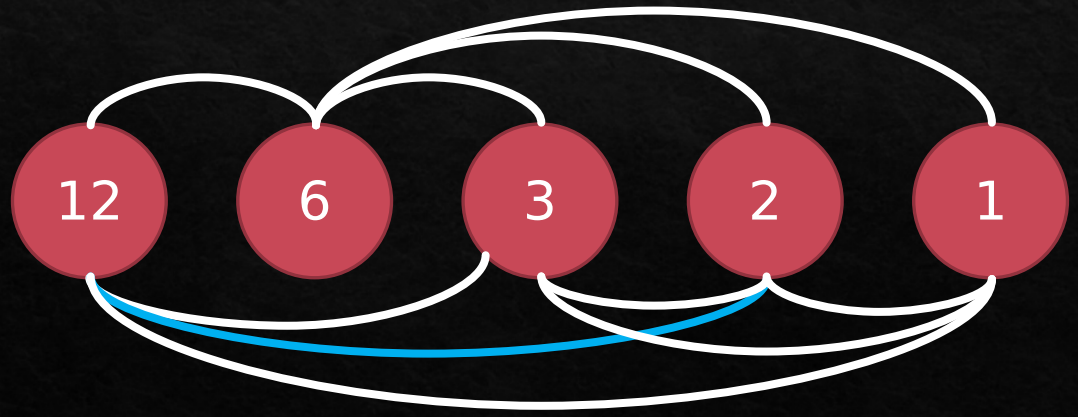
	Красный	Жёлтый	Зелёный
Красный			+
Жёлтый			
Зелёный	+		

Изображение

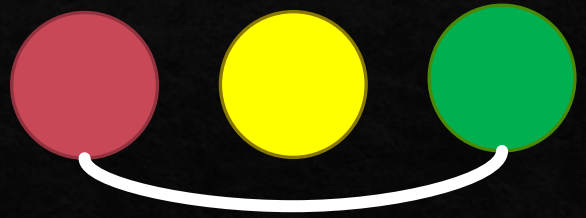
V_1={a, b, c, d}
E_1={{a,b},{b,c},{c,d},{a,c},{b,d}}



V_2={12, 6, 2, 3, 1}
E_2={{12,6}, {12,3}, {12,2}, {12,1}, {6,2}, {6,3}, {6,1}, {2,1}, {3,1}}



V_4={Красный, Жёлтый, Зелёный}
E_4={{Красный, Зелёный}}

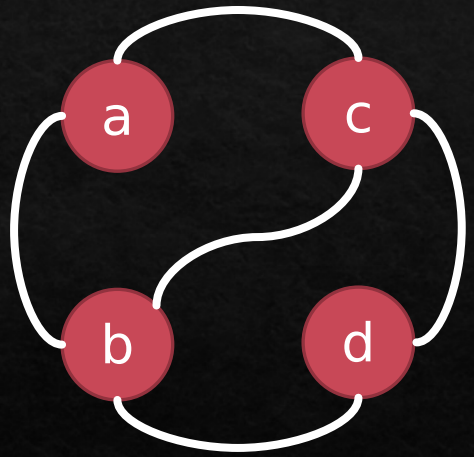


Изоморфизм графов

G

$V(G)$={a, b, c, d}
$E(G)$={{a,b}, {b,c}, {c,d}, {a,c}, {b,d}}

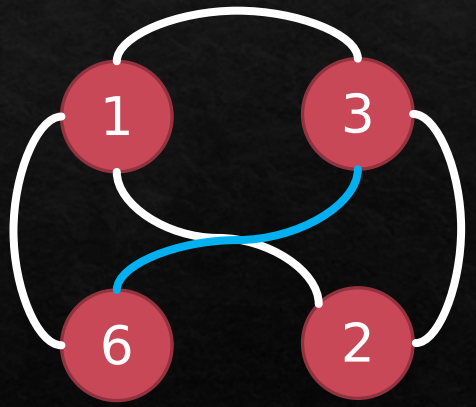
	a	b	c	d
a		+	+	
b	+		+	+
c	+	+		+
d		+	+	



H

$V(H)$={1, 2, 3, 6}
$E(H)$={{1,6}, {1,3}, {1,2}, {2,3}, {3,6}}

	1	2	3	6
1		+	+	+
2	+		+	
3	+	+		+
6	+		+	



Графы, определённый на различных множествах, сложно назвать «равными», но они могут иметь одинаковую структуру, которая позволит переносить результаты анализа с одного графа на другой.

Изоморфизм

Два графа G и H называются изоморфными, если существует такое взаимно однозначное отображение $M: V(G) \mapsto V(H)$ (переименование, подстановка), что $\forall u, v \in V(G)$ они смежны тогда и только тогда, когда смежны $M(u)$ и $M(v)$.

Можно доказать, что отношение изоморфности является отношением эквивалентности, что разбивает множество всех графов на непересекающиеся классы.

Легко показать, что два графа являются изоморфными, если матрица смежности одного графа может быть получена из матрицы смежности другого графа путём перестановки столбцов и/или строк.

G

$V(G)$={a, b, c, d}
$E(G)$={{a,b}, {b,c}, {c,d}, {a,c}, {b,d}}

	a	b	c	d
a		+	+	
b	+		+	+
c	+	+		+
d		+	+	

	b	a	c	d
b		+	+	+
a	+		+	
c	+	+		+
d	+		+	

H

$V(H)$={1, 2, 3, 6}
$E(H)$={{1,6}, {1,3}, {1,2}, {2,3}, {3,6}}

	1	2	3	6
1		+	+	+
2	+		+	
3	+	+		+
6	+		+	

M:

a	b	c	d
2	1	3	6

Изоморфизм

Благодаря изоморфизму мы можем выполнять вычисления над некоторым графом, переименовав его вершины как нам удобно. Это позволяет абстрагироваться от конкретных множеств при построении алгоритмов.

Методы хранения графов в памяти

Перечисление

V={a, b, c, d}
E={{a,b},{b,c},{c,d},{a,c},{b,d}}

```cpp
#include <iostream>

struct Edge {
        unsigned u, v;
    };

int main() {
    // a -> 1, b -> 2, c -> 3, d -> 4
    Edge edges[5] = {Edge{1, 2}, Edge{1, 3}, Edge{2, 3}, Edge{2, 4}, Edge{3, 4}};

    return 0;
}
```

Непосредственно множество вершин мы не создаём. Каждая вершина графа характеризуется индексом.

Списки инцидентности

```cpp
#include <iostream>

struct Edge {
    unsigned u, v;
};

struct EdgeListNode {…

struct Vertex {…

EdgeListNode* list_push_front(EdgeListNode *head, Edge e) {…

int main() {
    // a -> 1, b -> 2, c -> 3, d -> 4
    Vertex vertices[5];
    vertices[1].incident_list =
        list_push_front(
            list_push_front(nullptr, Edge{1, 2}), Edge{1, 3});
    vertices[2].incident_list =
        list_push_front(
            list_push_front(
                list_push_front(nullptr, Edge{2, 4}), Edge{2, 3}), Edge{2, 1});
    vertices[3].incident_list =
        list_push_front(
            list_push_front(
                list_push_front(nullptr, Edge{3, 4}), Edge{3, 2}), Edge{3, 1});
    vertices[4].incident_list =
       list_push_front(
            list_push_front(nullptr, Edge{4, 2}), Edge{4, 3});

    return 0;
}
```

{a, b}, {a, c}
{b, a}, {b, c}, {b, d}
{c, a}, {c, b}, {c, d}
{d, b}, {d, c}

Списки смежности

```cpp
#include <iostream>

struct AdjacentList {
    unsigned v;
    AdjacentList *tail;
};

AdjacentList* list_push_front(AdjacentList *head, unsigned v) {…

int main() {
    // a -> 1, b -> 2, c -> 3, d -> 4
    AdjacentList* vertices[5];
    vertices[1] =
        list_push_front(
            list_push_front(nullptr, 2), 3);
    vertices[2] =
        list_push_front(
            list_push_front(
                list_push_front(nullptr, 4), 3), 1);
    vertices[3] =
        list_push_front(
            list_push_front(
                list_push_front(nullptr, 4), 2), 1);
    vertices[4] =
       list_push_front(
            list_push_front(nullptr, 2), 3);

    /*for (unsigned v_idx = 1; v_idx != 5; ++v_idx) {…

    return 0;
}
```

{b, c}
{a, c, d}
{a, b, d}
{b, c}

Матрица смежности

```cpp
#include <iostream>

int main() {
    // a -> 0, b -> 1, c -> 2, d -> 3
    bool Adj[4][4] = {
        {0, 1, 1, 0},
        {1, 0, 1, 1},
        {1, 1, 0, 1},
        {0, 1, 1, 0}
    };

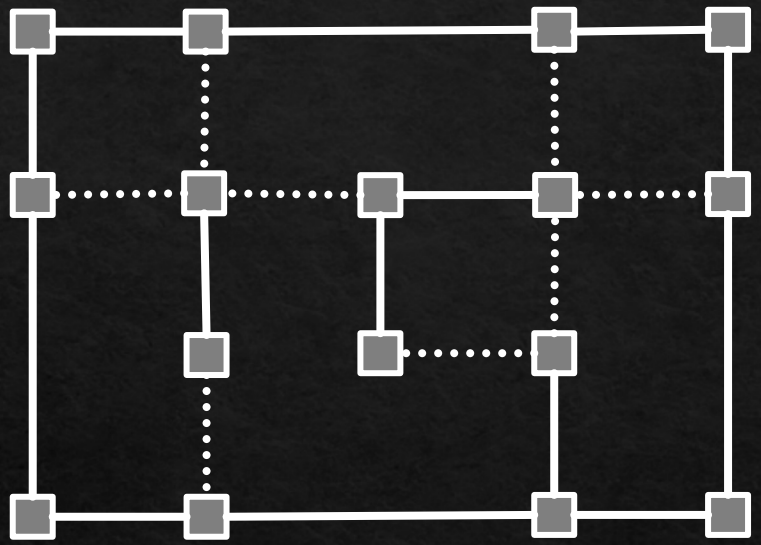
    return 0;
}
```

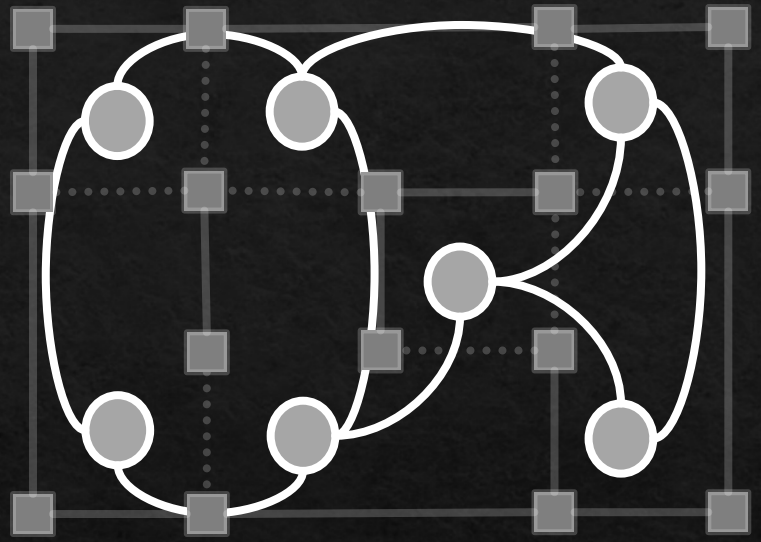
	a	b	c	d
a		+	+	
b	+		+	+
c	+	+		+
d		+	+	

Обход графа

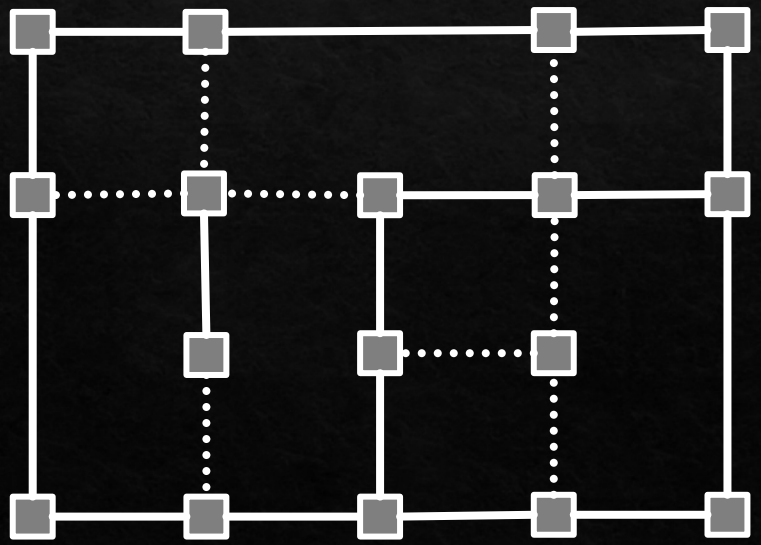
Поиск в глубину (bfs), поиск в ширину (bfs)

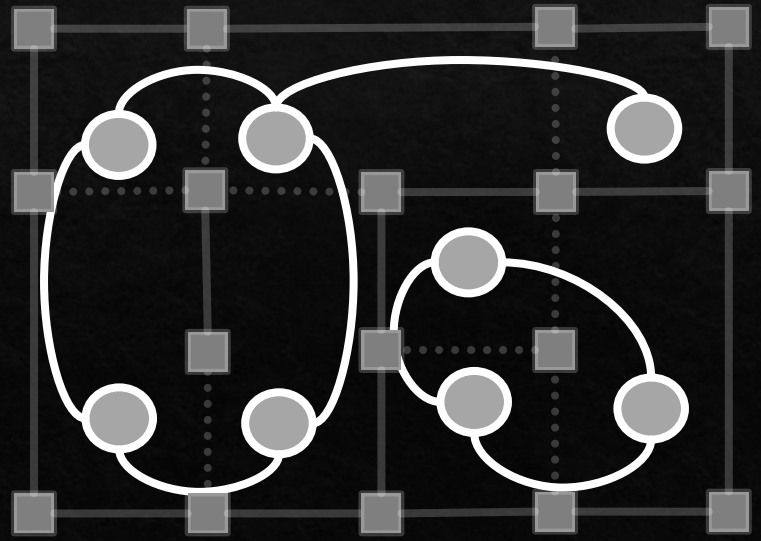
Можно ли посетить все комнаты?





Можно





Нельзя

Определения

Маршрутом в графе называется чередующаяся конечная или бесконечная последовательность вершин и рёбер $v_0(e_0)v_1(e_1)\dots(e_{n-1})v_n$, что $e_k = \{v_k, v_{k+1}\}$. Вершины v_0 и v_n называются крайними, остальные внутренними (промежуточными).

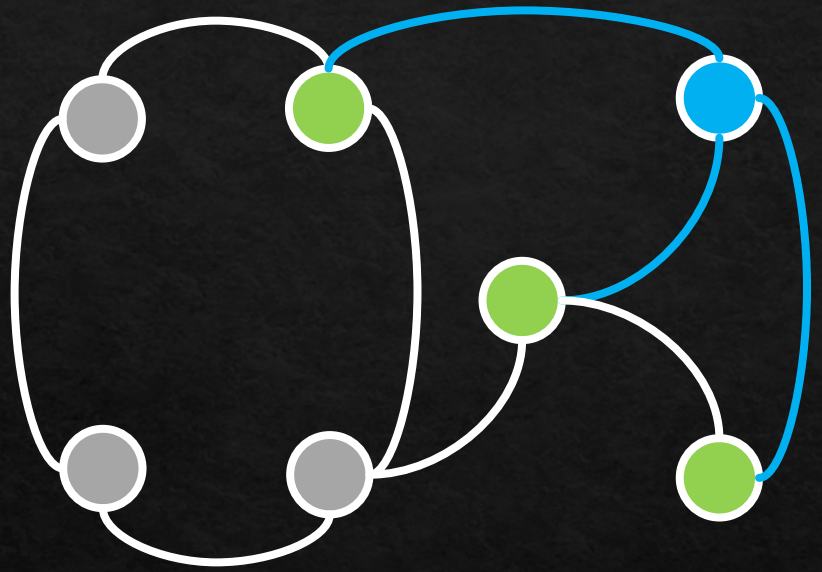
Вершина v_l называется достижимой из вершины $v_k \neq v_l$, если существует конечный маршрут для которого вершины v_k и v_l являются крайними. Для неориентированного графа отношение достижимости симметрично.

Связным называется граф, в котором любые две вершины достижимы.

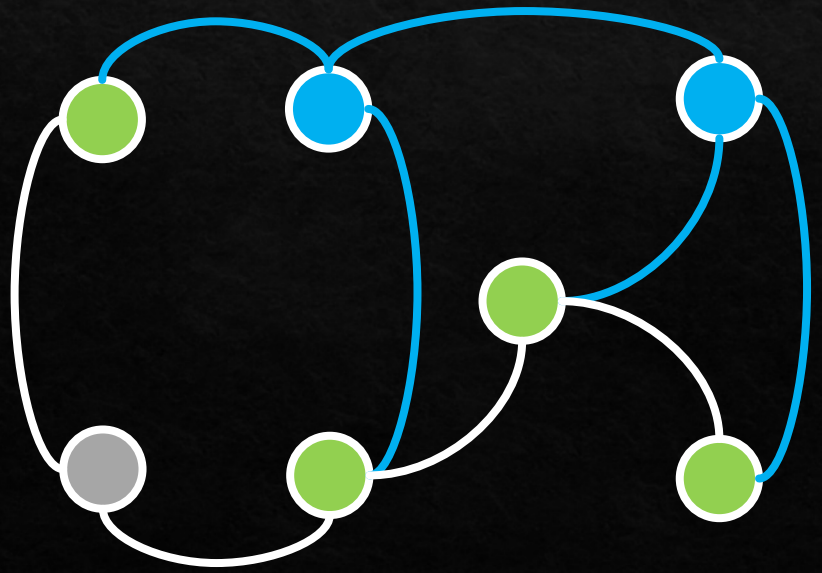
Компонентой связности называется максимальный (по включению вершин и рёбер) связный подграф.

Обход в глубину (Depth First Search)

Задача: посетить все достижимые вершины.



Начнём с некоторой вершины и отметим посещёнными все смежные вершины и инцидентные рёбра.

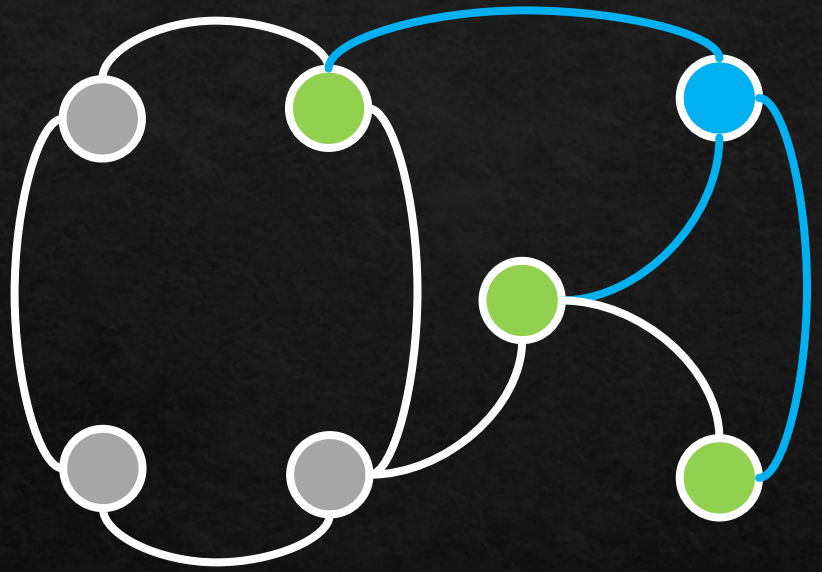


Повторим процедуру для всех смежных вершин, выбирая каждый раз последнюю добавленную.

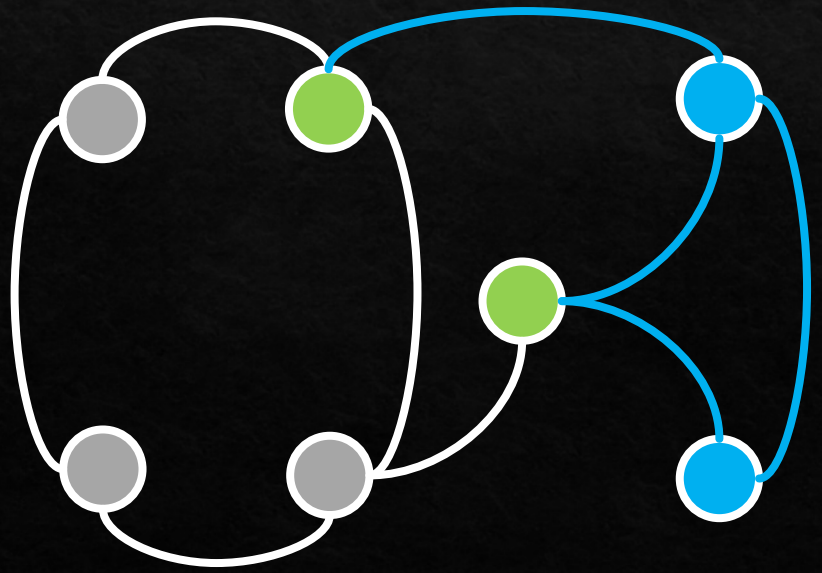
Для реализации можно использовать стек вершин.

Обход в ширину (Breadth First Search)

Задача: посетить все достижимые вершины.



Начнём с некоторой вершины и отметим посещёнными все смежные вершины и инцидентные рёбра.



Повторим процедуру для всех смежных вершин, начиная с первой добавленной.

Для реализации можно использовать очередь вершин.

Минимальное остовное дерево

Дерево, взвешенный граф, Жадные алгоритмы Крускала и Прима.

Дерево

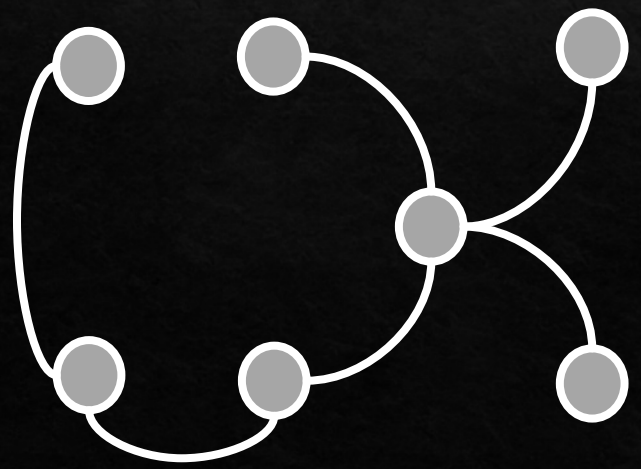
Цепью называется маршрут, в котором все рёбра различны.

Циклом называется цепь, в которой крайние вершины совпадают (замкнутая цепь).

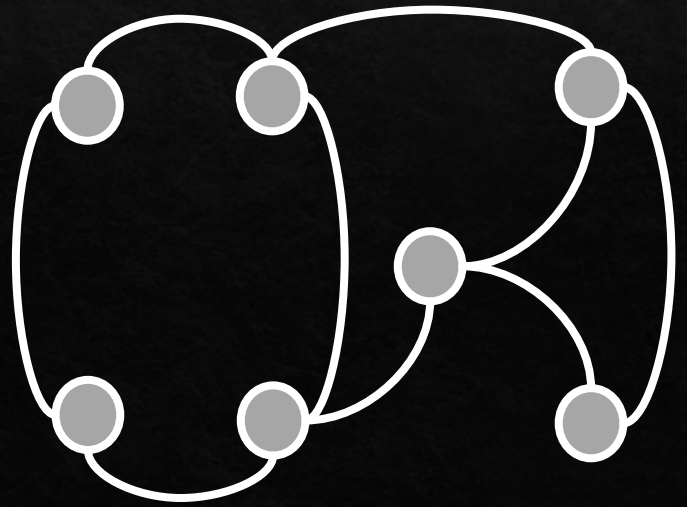
Граф называется ациклическим, если в нём нет циклов.

Деревом называется связный ациклический граф.

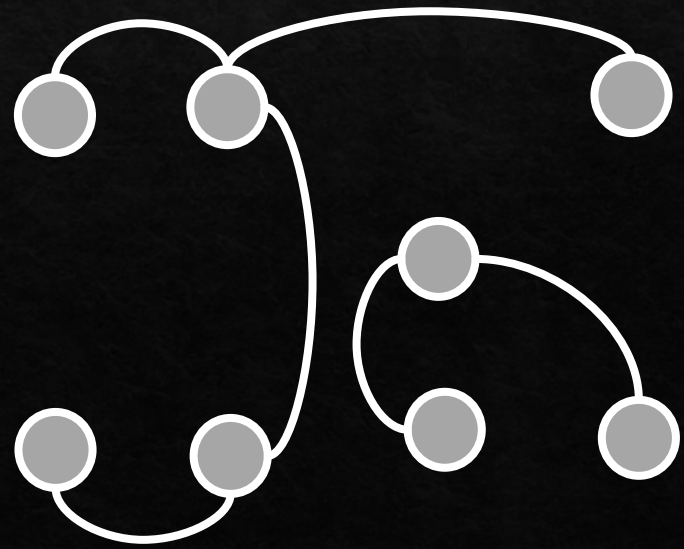
Дерево



Не дерево



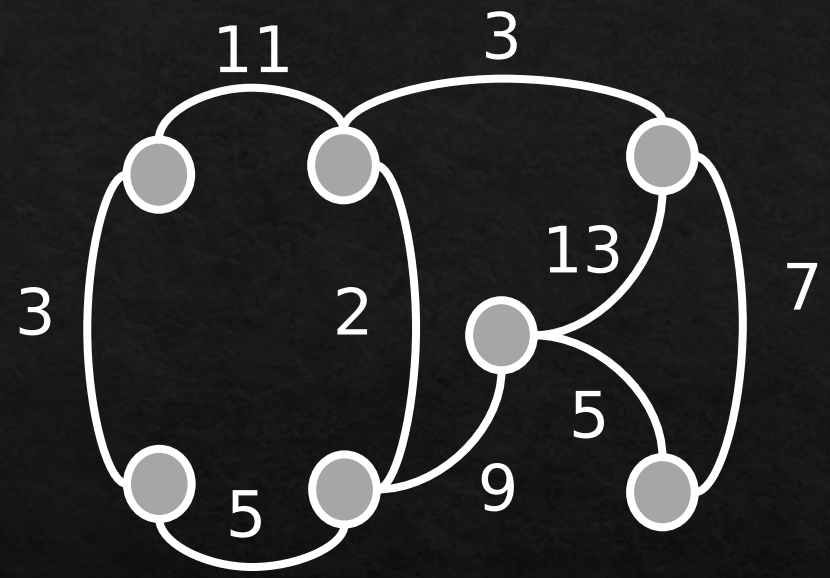
Не дерево



Взвешенный граф и минимальное остовное дерево

Взвешенным называется граф, в котором каждому ребру присвоено некоторое число - вес.

$e = \{u, v, w\}, u, v \in V, w \in \mathbb{R}$



Минимальным остовным деревом связного взвешенного графа G называется подграф $\acute{G}$, который включает все вершины G, является деревом и обладает минимальной суммой весов всех его рёбер.

Если все веса в графе различны, то минимальное остовное дерево единственно.

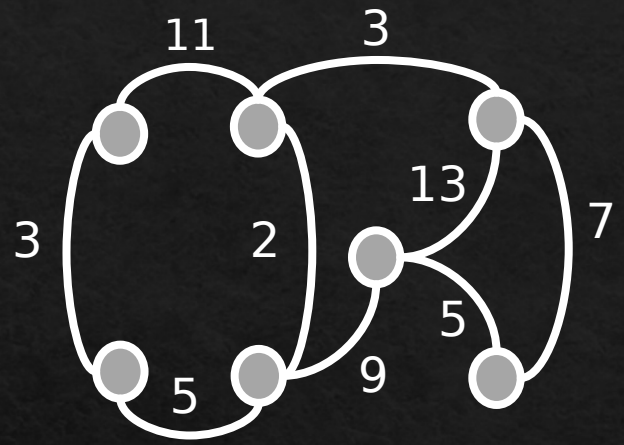
Крускал

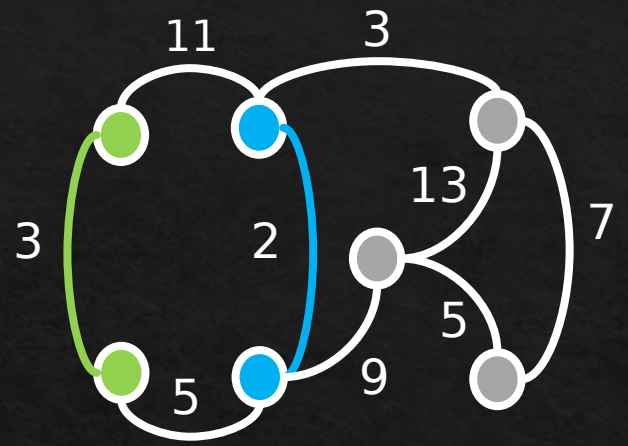
1. Отсортируем все рёбра по весу.
2. На каждом шаге будем добавлять к дереву самое минимальное по весу ребро из оставшихся, следуя следующим правилам:
   a. если ребро соединяет две не отмеченные вершины, то поместим их в новое множество вершин, если одна из вершин принадлежит одному из множества отмеченных, то добавим эту вершину к множеству;
   b. если ребро соединяет две вершины из одного множества, пропустим его;
   c. если ребро соединяет две вершины из разных множеств, то объединим их.
3. Алгоритм завершает работу, когда мы перебрали все рёбра.

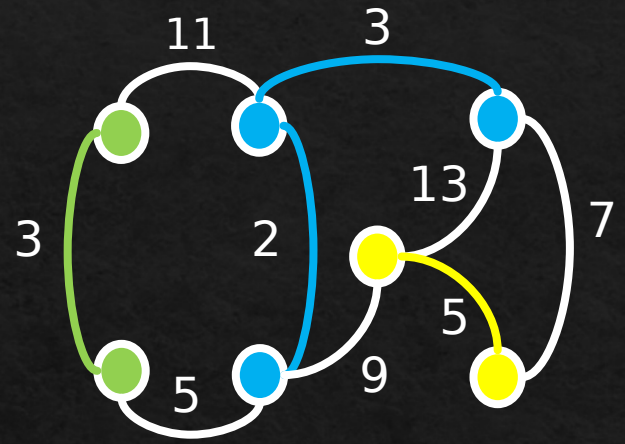
Алгоритмическая сложность алгоритма зависит от того, как организованы множества вершин:

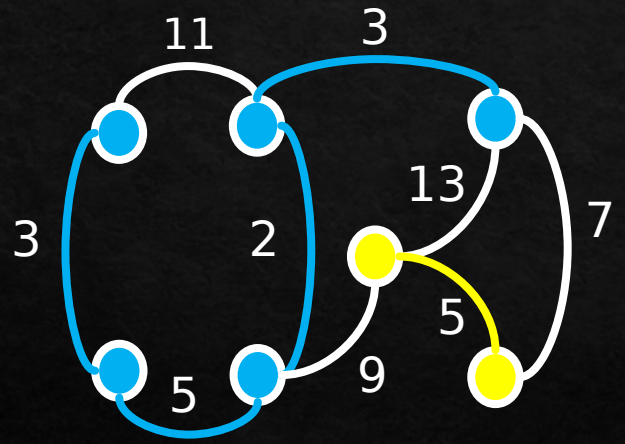
$O(Elog(E) + E^2)$ При объединении множеств нужно проходится по всем элементам и перекрашивать их.

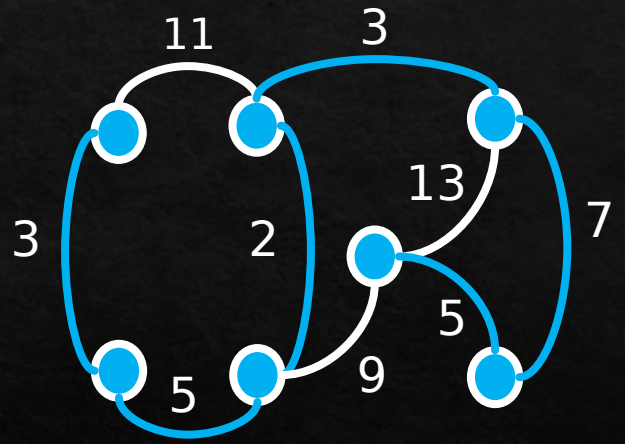
$O\big(Elog(E)\big)$ Используется ссылочная система непересекающихся множеств.

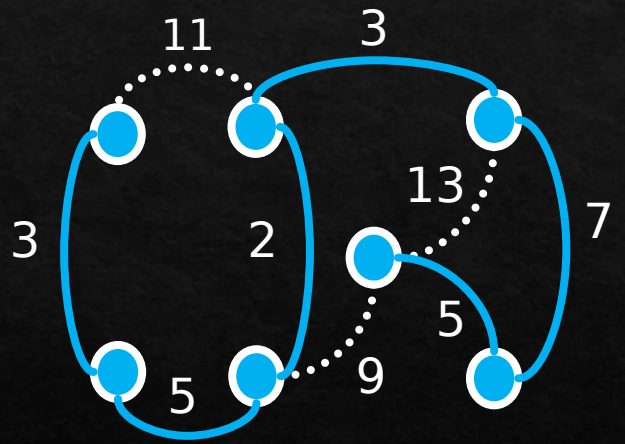

11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5

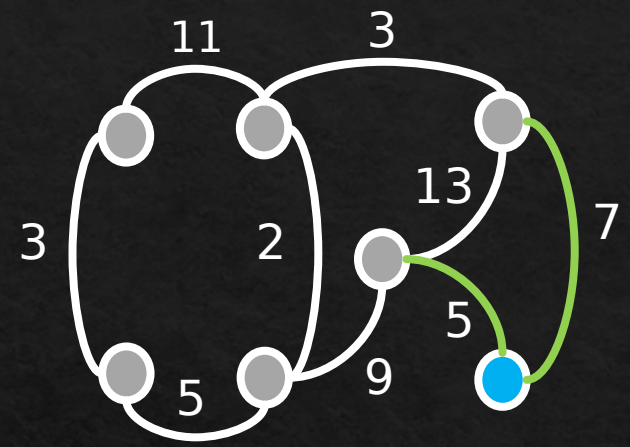

11
3
13
7
3
2
5
9
5

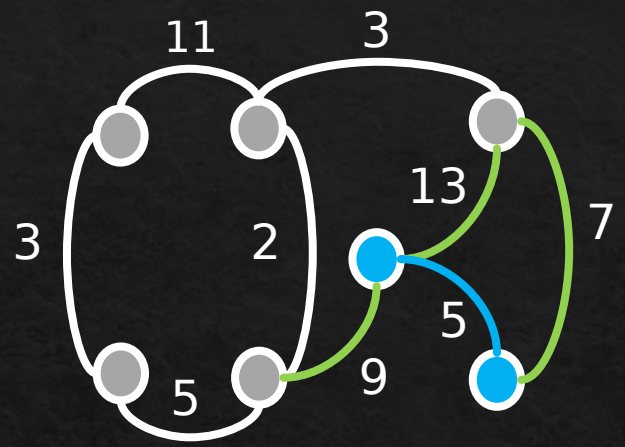
Прим

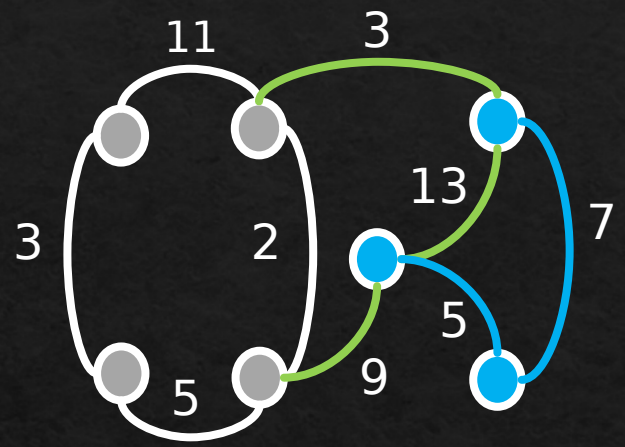
1. Выберем произвольную вершину и отметим её как добавленную.
2. На каждом шаге будем добавлять новое ребро по следующим правилам:
   a. из всех рёбер, которые ведут из множества отмеченных вершин выберем самое минимальное;
   b. если ребро ведёт в отмеченную вершину – выкинем его;
   c. если ребро ведёт в неотмеченную вершину – добавим ребро и отметим вершину.
3. Процесс завершается, если мы отметили все вершины.

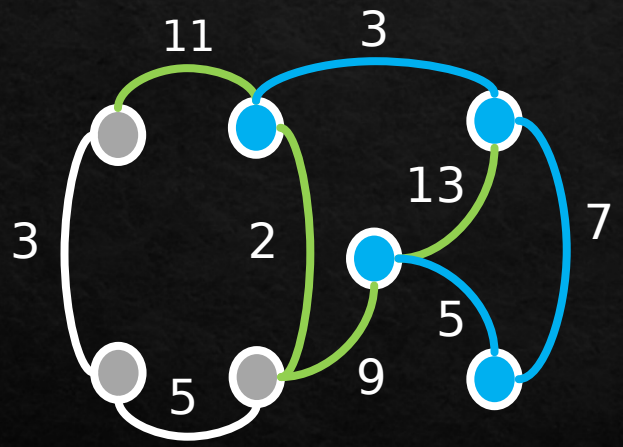
Алгоритмическая сложность при использовании очереди с приоритетом на рёбрах:

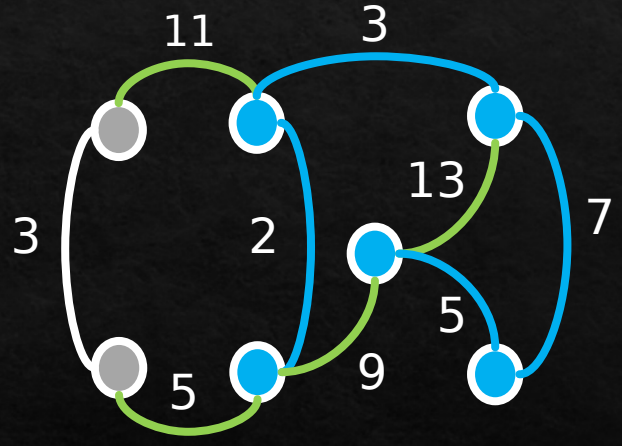
$$O\big((E+V)log(V)\big)$$

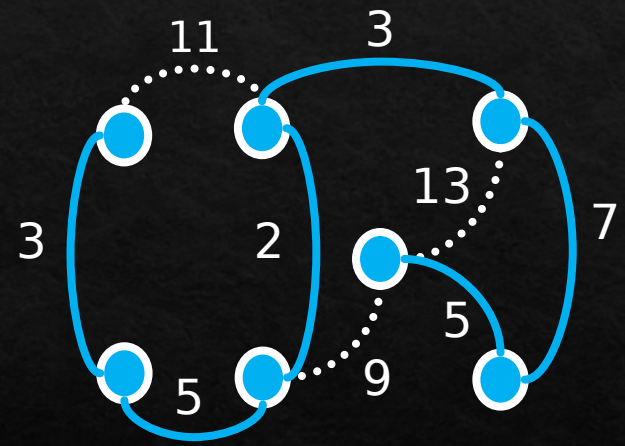

11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5


11
3
13
7
3
2
5
9
5